

ASTRALIS

Ingeniería de Software II

UNAL 2026-1S

Profesor: Sergio E. Vargas P.

sevargasp@unal.edu.co

Descripción

El **Observatorio Astronómico Nacional** de la UNAL, fundado en 1803, es el observatorio científico más antiguo de Colombia y uno de los más importantes de Latinoamérica. Lleva décadas rastreando el cielo desde Bogotá. Su software, sin embargo, sigue siendo un problema abierto.

Este semestre ustedes lo resuelven.

ASTRALIS es una plataforma web que permite simular, visualizar y predecir fenómenos astronómicos usando datos reales de la NASA. El sistema corre simulaciones físicas de cuerpos celestes, expone un modelo de Machine Learning para clasificar exoplanetas, y todo eso vive desplegado en la nube con un pipeline CI/CD que nunca duerme.

No es un ejercicio académico aislado. Es un sistema de producción con las mismas preocupaciones de cualquier producto profesional en Colombia o en el exterior: escalabilidad, seguridad, automatización y ética de software. Las mismas herramientas que usan equipos en Rappi, Bancolombia, Platzi o cualquier startup tecnológica de la región.

El equipo: 4 roles, 1 sistema

Son 4 integrantes. Cada uno lidera un módulo técnico con responsabilidad real. No hay tareas de relleno. Todo lo que construya cada uno es consumido directamente por los demás: si el backend cae, el frontend no funciona. Si el pipeline falla, nadie despliega. Eso es ingeniería real.

E1 • Backend Engineer – "El arquitecto del núcleo" Diseña y construye la API REST. Es el contrato que todos los demás consumen. Maneja autenticación, seguridad y documentación. Si su API cae, el sistema cae. Stack sugerido: FastAPI • PostgreSQL • JWT • OAuth2 • Swagger/OpenAPI

E2 • DevOps Engineer – "El guardián del pipeline" Construye la infraestructura que permite que el código de todos llegue a producción de forma automática y confiable. Sin su trabajo, el sistema solo existe en los computadores del equipo. Stack sugerido: GitHub Actions • Docker • Railway • Grafana Cloud • Docker Compose

E3 • ML & Physics Engineer – "El cerebro científico" Entrena el modelo con datos reales del telescopio Kepler y construye el simulador N-cuerpos que da vida a las órbitas planetarias. Su trabajo tiene nombre: predecir si un objeto en el cielo es un planeta o no. Stack sugerido: scikit-learn • Python • rebound • Jupyter Notebooks • NASA APIs

E4 • Frontend Engineer – "El que enamora al usuario" Construye la interfaz que convierte datos crudos de la NASA en algo que cualquier persona pueda explorar y entender. Es el módulo que verá el mundo en el demo day. Stack sugerido: React.js • Three.js • Canvas API • Vite

Los datos: reales, públicos y gratuitos

NASA Exoplanet Archive – exoplanetarchive.ipac.caltech.edu Más de 9.000 candidatos a exoplanetas del telescopio Kepler. El dataset principal del modelo ML.

JPL Horizons System – ssd.jpl.nasa.gov/horizons Posiciones y velocidades de cualquier cuerpo del sistema solar en tiempo real. El motor del simulador.

Open Notify ISS API – open-notify.org/Open-Notify-API Posición de la Estación Espacial Internacional actualizada cada segundo. Perfecto para el demo en vivo.

JPL Small-Body Database – ssd.jpl.nasa.gov/sbdb.cgi Parámetros orbitales de todos los asteroides conocidos, incluidos los de acercamiento a la Tierra.

Las fases del semestre

Fase 1 • Diseño y arquitectura

Nadie escribe código de producto todavía. Todo el esfuerzo va en diseño. Las decisiones que tomen en estas cuatro semanas las van a vivir el resto del semestre, para bien o para mal.

- **E1:** Diseño de la API: endpoints, modelos de datos, diagrama Entidad-Relación, contratos de autenticación.
- **E2:** Repositorio monorepo en GitHub con ramas protegidas, reglas de PR y esqueleto del pipeline CI.
- **E3:** Exploración del dataset NASA Exoplanet Archive. Prototipo de simulación N-cuerpos en Jupyter Notebook.
- **E4:** Wireframes de las pantallas principales. Decisión de stack. Componentes base sin lógica de negocio.

Entregable: Documento de arquitectura con diagrama C4 nivel 2 + repositorio configurado + mínimo 3 Architecture Decision Records (ADR) justificados.

Fase 2 • Construcción del núcleo

El sistema cobra vida. Es la primera vez que los 4 módulos se integran y algo funciona de punta a punta.

- **E1:** API REST funcional con al menos 6 endpoints, autenticación JWT, validaciones básicas contra OWASP Top 10.
- **E2:** Pipeline CI/CD completo: lint → tests → build → deploy automático a staging en cada merge a main. Cobertura de tests mínima del 60%.
- **E3:** Modelo ML entrenado y evaluado – clasificación de exoplanetas: confirmado o falso positivo. Simulador N-cuerpos expuesto como módulo que la API puede llamar.
- **E4:** Pantallas principales funcionales: simulador orbital 2D animado, catálogo de exoplanetas con filtros, visualización de curvas de luz de tránsito.

Entregable: Demo end-to-end funcional en staging. Parcial 1: code review cruzado entre compañeros + pipeline corriendo en vivo durante la sesión de clase.

Fase 3 • Seguridad, calidad y ML en producción

El sistema se endurece. Y llega el momento más tenso del semestre: el **hacking ético cruzado**. Cada estudiante intenta vulnerar el módulo de un compañero distinto y entrega un reporte técnico de lo que encontró.

- **E1:** Auditoría OWASP Top 10 sobre la API. Rate limiting, sanitización de inputs, headers de seguridad. Documentación Swagger/OpenAPI completa.
- **E2:** Monitoreo activo con alertas. Logs centralizados. Runbook del pipeline documentado.
- **E3:** Modelo servido como microservicio independiente con versionado. Métricas documentadas: accuracy, F1, matriz de confusión. Análisis de sesgos del dataset.
- **E4:** Visualización 3D del sistema solar con Three.js. UX responsive. Integración completa con todos los endpoints.

Entregable: Parcial 2 – reporte técnico de las vulnerabilidades encontradas en el módulo asignado y las correcciones aplicadas.

Fase 4 • Integración final y demo day

El sistema vive en producción. El equipo lo defiende ante el curso. Si el sistema cae durante el demo, el postmortem de lo que falló hace parte de la evaluación. Caerse no descuenta automáticamente. Ocultarlo sí.

- **E1:** Tests de integración y carga con k6 o Locust. Documentación técnica final de la API.
- **E2:** Deploy a producción confirmado. Dashboard de monitoreo activo durante la presentación. Postmortem escrito del pipeline.
- **E3:** Reflexión ética documentada: ¿qué implicaciones tiene que el modelo clasifique mal un objeto potencialmente peligroso? ¿Cómo se mitiga?
- **E4:** Demo day: presentación pública del sistema funcionando en producción.

Entregable final: Sistema desplegado y accesible • Video demo de 5 minutos • README completo con instrucciones de instalación local • Presentación oral de 20 minutos.

Stack tecnológico

Todo gratuito, todo de uso profesional en la industria:

Capa	Tecnologías
Backend	FastAPI o Node.js + Express · PostgreSQL · JWT + OAuth2
DevOps	GitHub Actions · Docker · Railway / Render / Fly.io
ML y Física	scikit-learn · Python · rebound · Jupyter
Frontend	React.js · Three.js · Canvas API · Vite
Calidad	pytest · k6 · OWASP ZAP · ESLint · Cobertura 60%+
Monitoreo	UptimeRobot · Grafana Cloud

Lo que no entra este semestre

Un buen ingeniero sabe qué no construir. Estas decisiones ya están tomadas:

- Procesamiento de datos LIGO / ondas gravitacionales
 - GraphQL – REST es suficiente para todos los casos de uso
 - Despliegue en AWS/GCP con costo
 - Aplicación móvil nativa
 - Visualización 3D avanzada con física compleja – es un bonus opcional
-

Universidad Nacional de Colombia · Departamento de Ingeniería de Sistemas e Industrial · Sede Bogotá · 2026 "Inter Aulas Academiae Quære Verum"